

PES UNIVERSITY

Department of Computer Science and Engineering
Software Engineering (UE24CS341A) · Semester 5 · Section C

Lab 1 – Requirements Engineering & UML Use-Case Modelling

Problem Statement #31: Multi-Vendor Artisan E-Commerce Marketplace

Name	Diya R Gowda
SRN	PES1UG24CS159
Section / Sem	C / 5
Submitted	27 August 2026

Deliverables in this document: Requirements Table (FR-001–FR-005, NFR-001–NFR-002), UML Use-Case Diagram, and a one-page Use-Case Flow Specification with a supporting flowchart. Both diagrams were drawn in draw.io (diagrams.net); the editable .drawio source files are submitted alongside this PDF.

1. Scenario, in my own words

Problem Statement #31 describes an online marketplace for independent craftspeople. Each artisan runs their own storefront on the platform, lists the items they make, and receives orders from shoppers who may buy from several artisans at the same time. When a shopper pays, the platform has to divide that one payment between the vendors whose items are in the cart, keep a 5% commission for itself, and pass the rest to the right vendor accounts. The catalogue is image-heavy, so pages have to stay fast even with lots of product photos.

I worked from the two samples given in the problem statement (the payment-split FR and the catalogue-performance NFR) and kept them as FR-001 and NFR-001, then added FR-002 to FR-005 and NFR-002 by going through the scenario and listing what a shopper and a vendor each need the system to do.

2. Actors and Use Cases

Actors (3)

- **Shopper** – browses the catalogue, manages a cart, checks out, and tracks orders.
- **Artisan Vendor** – sets up a storefront, manages listings, fulfils orders, and receives payouts.
- **Payment Gateway** – external system that authorises the card payment and carries out the split transfers to the vendor and platform accounts.

Use Cases (8)

- UC-01 Register / Sign In
- UC-02 Manage Storefront & Listings
- UC-03 Search & Browse Catalogue
- UC-04 Manage Cart
- UC-05 Checkout & Place Order (core use case for the flow below)
- UC-06 Process Split Payment
- UC-07 Fulfil Order & Ship
- UC-08 Release Vendor Payout

Supporting use cases used by the «include» / «extend» relations: Validate Stock & Reserve Items, Deduct 5% Platform Commission, Apply Discount Code, Retry Payment.

3. Requirements Table

Exactly five functional requirements and two non-functional requirements. FR-001 and NFR-001 are the two given in Problem Statement #31, rephrased in my own words.

Req ID	Type	Description ("The system shall ...")	Priority	Acceptance Criteria (pass / fail)	Rationale
FR-001	Functional	The system shall split a Shopper's cart payment at checkout, sending each item's earnings to the correct Artisan Vendor account after taking a 5% platform commission.	High	For a cart holding items from three vendors, the three vendor payouts plus the platform commission add up to the exact amount charged to the Shopper (checked to the paisa). If the parts do not add up, the split is rejected and logged.	This is the main reason the marketplace exists. Vendors have to be paid the right amount without anyone doing it by hand.
FR-002	Functional	The system shall let an Artisan Vendor create one storefront and publish product listings, where every listing has a title, description, price, available quantity and at least one image.	High	Saving a listing that has all required fields makes it show up in the catalogue within 10 seconds. Trying to save a listing with no price or no image is blocked and an inline error is shown next to the missing field.	There is nothing to sell until vendors can list items, and the required fields keep listings usable for shoppers.
FR-003	Functional	The system shall let a Shopper search the catalogue by keyword and narrow the results by category and price range, showing only listings that are in stock.	Medium	Searching "ceramic mug" returns listings whose title or description contains those words, ordered by relevance, with out-of-stock items left out. Setting a price filter of 200 to 500 removes every result priced outside that band.	Shoppers leave quickly if they cannot find what they want, and showing sold-out items wastes their time.
FR-004	Functional	The system shall let a Shopper check out a cart that contains items from more than one vendor in a single payment, and shall create a separate order record for each vendor.	High	Checking out a cart with items from two vendors results in one payment charge and two order records, each showing only that vendor's items and sub-total. Each vendor can see only their own order.	Shoppers expect to pay once, while each vendor needs their own order to pack and post.

Req ID	Type	Description ("The system shall ...")	Priority	Acceptance Criteria (pass / fail)	Rationale
FR-005	Functional	The system shall notify each Artisan Vendor when a new paid order arrives and shall let the vendor mark the order as shipped, which releases the held payout for that order.	Medium	Within one minute of a successful payment the vendor gets an in-app notification and an email. Marking the order "Shipped" changes its payout status from Held to Released and stores a timestamp. The payout cannot move to Released before payment is confirmed.	Vendors need to know an order is waiting, and holding the payout until dispatch protects the shopper if the item never ships.
NFR-001	Non-functional (Performance)	The product catalogue shall serve listing images through a CDN so that a catalogue page loads in under 500 ms for at least 95% of requests.	High	A load test of 1,000 catalogue-page requests on a normal broadband connection shows the 95th-percentile full-page load time is below 500 ms, and every image request is answered by the CDN rather than the origin server.	Image-heavy pages that load slowly are the biggest reason shoppers give up on browsing.
NFR-002	Non-functional (Security)	The system shall limit each Artisan Vendor to their own orders, payouts and shopper shipping details, and shall never show a shopper's full card number to anyone.	High	Signed in as Vendor A, every attempt to open a Vendor B order or payout URL returns HTTP 403. Card numbers appear only as the last four digits on every screen, receipt and API response. A security scan finds no endpoint that returns another vendor's data.	A marketplace holds data for many separate parties. One vendor seeing another's sales, or a customer's card, breaks trust and very likely the law.

Peer critique & revisions made

During the peer-review step a classmate reviewed my table. Three things changed as a result:

- FR-003 first said only "search the catalogue". It was not testable, so I added the category and price-range filters and the rule that out-of-stock items are hidden, which gives a clear pass/fail.
- FR-001 originally did not say what happens when the split does not add up. I added that the split is rejected and logged, so the acceptance criterion has a fail case.
- NFR-002 was widened from "vendors should not see each other's data" to naming the exact response (HTTP 403) and the card-masking rule, so a tester knows precisely what to check.

4. UML Use-Case Diagram

Drawn in draw.io. Solid lines are associations. Dashed arrows are dependencies labelled «include» or «extend»; the arrow points from the base use case to the included one, and from an extending use case to the one it extends.

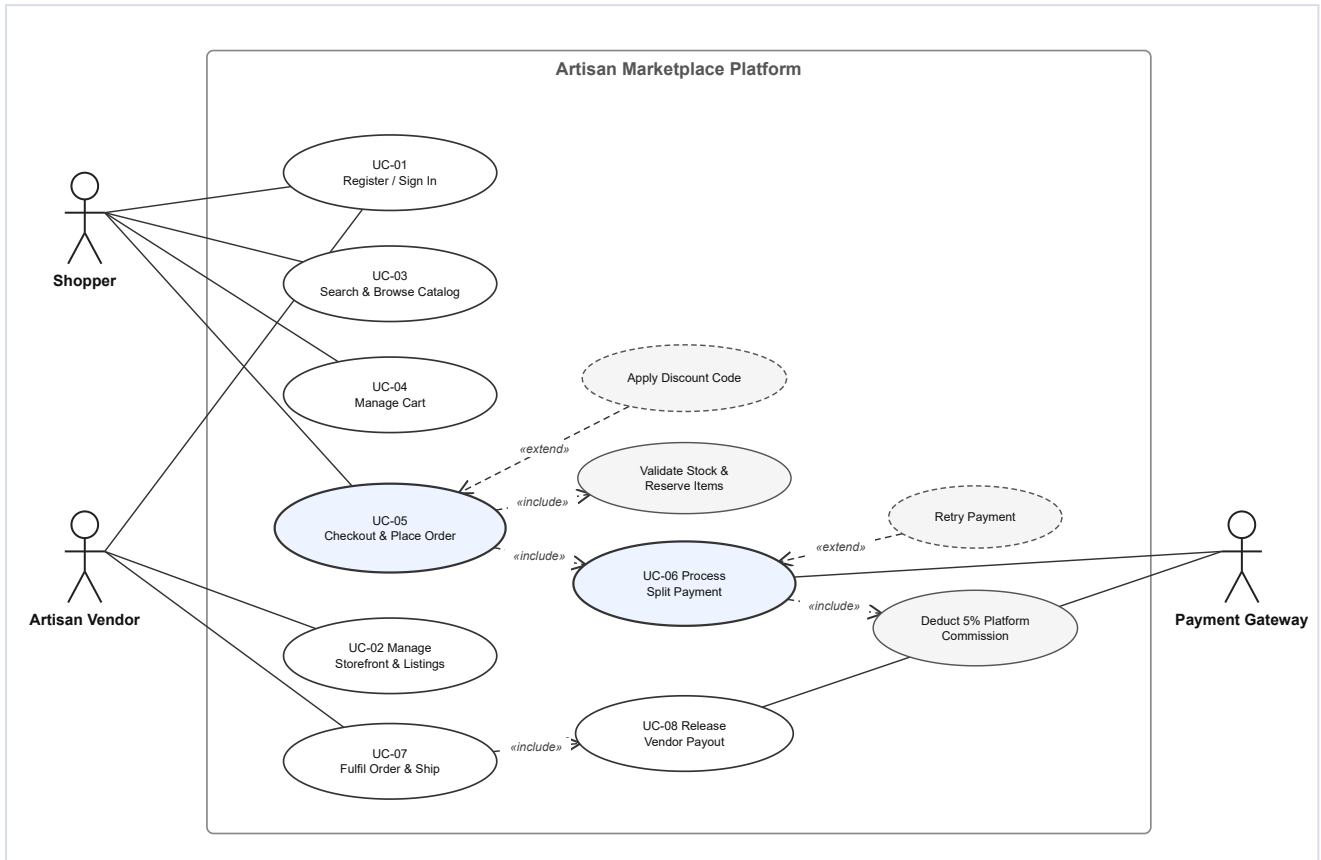


Figure 1 - Use-case diagram for the Multi-Vendor Artisan E-Commerce Marketplace (source: UseCase_Diagram.drawio).

Relationships shown

- **«include»:** UC-05 Checkout & Place Order includes *Validate Stock & Reserve Items* and UC-06 Process Split Payment; UC-06 includes *Deduct 5% Platform Commission*; UC-07 Fulfil Order & Ship includes UC-08 Release Vendor Payout. These steps always run as part of the base use case.
- **«extend»:** *Apply Discount Code* extends UC-05 (only when the shopper enters a code); *Retry Payment* extends UC-06 (only when the first attempt is declined).

5. Use-Case Flow Specification

Use Case: UC-05 Checkout & Place Order

Primary actor Shopper

Supporting actor Payment Gateway

Goal Pay once for a cart that may hold items from several vendors and get a confirmed order per vendor.

Preconditions

- The Shopper is signed in and the cart holds at least one in-stock item.
- Every cart item belongs to an active vendor storefront.
- The Shopper has a saved or newly entered payment method, and a shipping address on file or ready to enter.

Postconditions

- **On success:** one payment is captured from the Shopper; one order record per vendor is created with status *Paid - Awaiting Shipment*; each vendor payout is calculated as item earnings minus 5% and set to *Held*; stock for each ordered item is reduced; the cart is cleared and the Shopper sees a confirmation with a per-vendor breakdown.
- **On failure:** no payment is held (any authorisation is reversed); no order record is created; reserved stock is released; the cart is left as it was so the Shopper can try again.

Main Success Scenario

1. The Shopper opens the cart and selects "Checkout".
2. The system checks every cart item is still in stock and reserves those quantities. [«include» Validate Stock & Reserve Items]
3. The system shows an order summary grouped by vendor, with item earnings, the 5% platform commission line, shipping and the total.
4. The Shopper confirms the shipping address.
5. The Shopper selects a payment method and selects "Pay".
6. The system sends the total and the per-vendor split to the Payment Gateway. [«include» Process Split Payment]
7. The Payment Gateway authorises and captures the total, then transfers each vendor's share (earnings minus 5%) to their account and the commission to the platform account. [«include» Deduct 5% Platform Commission]
8. The system creates one order per vendor, sets each to *Paid - Awaiting Shipment*, and marks each payout *Held*.
9. The system reduces stock by the ordered quantities and clears the cart.
10. The system shows the confirmation page, emails the Shopper a receipt, and notifies each vendor of their new order.
11. The use case ends.

Alternate Flow – 7a. Payment declined

- 7a1. The Payment Gateway returns a "declined" result at step 7.

- 7a2. The system releases the stock it reserved at step 2 and shows "Payment failed, please try another method".
- 7a3. The Shopper enters a different payment method and selects "Pay" again. [«extend» Retry Payment]
- 7a4. If this second attempt succeeds, the flow continues from step 8.
- 7a5. If the second attempt also fails, the system cancels the checkout, keeps the cart intact, and returns the Shopper to the cart page. No order is created and no money is held.

Alternate Flow – 2a. Item sold out during checkout

- 2a1. At step 2 the system finds one cart item is no longer in stock.
- 2a2. The system marks that line "item unavailable", removes it from the order, and tells the Shopper what changed.
- 2a3. If at least one item remains, the Shopper can go on with the updated cart from step 3; otherwise the Shopper is returned to the cart and the use case ends with no order.

6. Supporting Flowchart – UC-05 Checkout & Place Order

Drawn in draw.io. This is the same flow as section 5, including both alternate flows (payment declined with a two-attempt limit, and an item going out of stock).

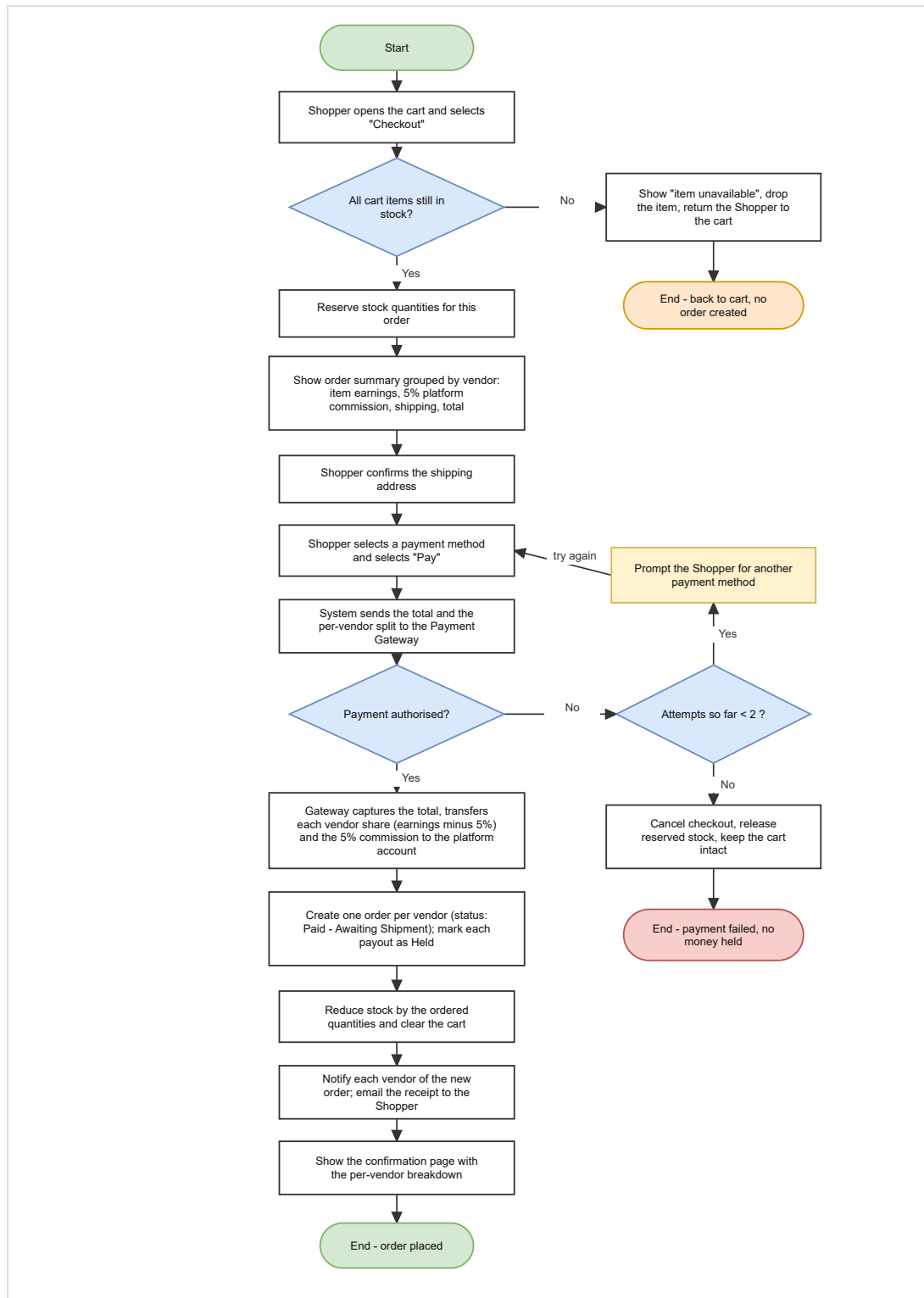


Figure 2 - Checkout & Place Order flowchart (source: Checkout_Flowchart.drawio).